



ISLAMIC UNIVERSITY OF TECHNOLOGY
A Subsidiary Organ of The Organisation of Islamic Cooperation (OIC)



Final Project Report

FeatherDB

Software Project Lab 1

SWE 4304

Third Semester, Session 2023-2024

Submitted to

Samia Nawsheen
Junior Lecturer

Submitted by

Md. Saif-Al Sarker
230042106

M Safwan Hasan Khan
230042117

Wasi Omar Syed
230042143

Date of Submission

30 March, 2026

Project Overview and Motivation

FeatherDB is a lightweight, file-based relational database engine written from scratch in **C++17**. It provides a transparent command-line interface (CLI) for interacting with data using SQL-like commands, specifically designed to demonstrate the internal mechanics of database systems without the overhead of massive, server-based RDBMS.

Motivation: The idea came from YouTube videos and "build your own X" repositories on GitHub; we wanted to learn by building something real rather than just studying how it worked in theory. Coming from a C background, a database engine felt like the right challenge. RBAC and the ERD visualizer were added on the recommendation of our supervisor and judges during the proposal phase.

Problem Statement

When we moved from our C programming course into DBMS, something felt off about SQL in a way that was hard to ignore. In C, you write out everything, you manage memory, you loop through arrays, and you explicitly decide how data gets shuffled around. You can read your own code and trace exactly what happens. SQL isn't like that. You write `SELECT * FROM users WHERE age > 18` and it just works. Something underneath figures out how to scan the table, apply the filter, and hand you the result. But what is that something?

That question stuck with us. Production databases like PostgreSQL and MySQL obviously have answers buried in them somewhere, but those codebases are millions of lines across decades of development, not something you crack open to learn from. So we were stuck in an uncomfortable position: using a tool we couldn't see inside, in a course that was supposed to be teaching us how it worked.

FeatherDB started as an attempt to answer that question for ourselves. Not to build something production-ready, not to compete with anything, just to find out what actually happens between typing a SQL command and seeing a result.

Potential Solution

The core idea behind FeatherDB was to build a database where every step is something we wrote and could therefore understand. The architecture follows a clean pipeline, a SQL string comes in, gets tokenized into recognizable pieces, gets parsed into an Abstract Syntax Tree that captures its meaning, and then gets handed to an executor that actually does the work. Writing that pipeline ourselves meant that "how does SQL turn into an operation" stopped being a mystery and became something we could point to in our own code.

From there, we wanted the database to actually behave like one, not just parse queries but enforce the things databases are supposed to enforce. So FeatherDB supports primary keys, unique constraints, and foreign keys with referential integrity. When you try to delete a row that another table depends on, it refuses. Writing that enforcement from scratch made relational integrity concrete in a way that just reading about it doesn't.

We also added support for nested subqueries, both in FROM clauses and WHERE clauses, because flattening a recursive query structure into something executable was exactly the kind of problem that seemed impossible until you sit down and work through it. Similarly, ORDER BY runs through an in-memory quicksort, which turned out to be a natural place where our C background actually helped us think clearly about what the database needs to do.

The RBAC system came out of a separate curiosity & advice from our supervisor: databases don't just store data, they control who can touch what. We wanted to understand that layer too. So FeatherDB has a secret-key authentication system, role creation, and per-role permissions on specific tables, all sitting as a validation step before any query reaches the storage engine. It's not enterprise security, but it made the concept of authentication and authorization at the database level tangible rather than abstract.

The ERD viewer was almost an afterthought, but it ended up being one of the more satisfying parts. You can define foreign keys across tables and then run `.erd` and see the 1:1, 1:N, and N:N relationships visualized in the terminal. Seeing the relationships you defined reflected back as a tabular diagram made it clear why schema design matters in a way that diagrams in a textbook don't quite capture.

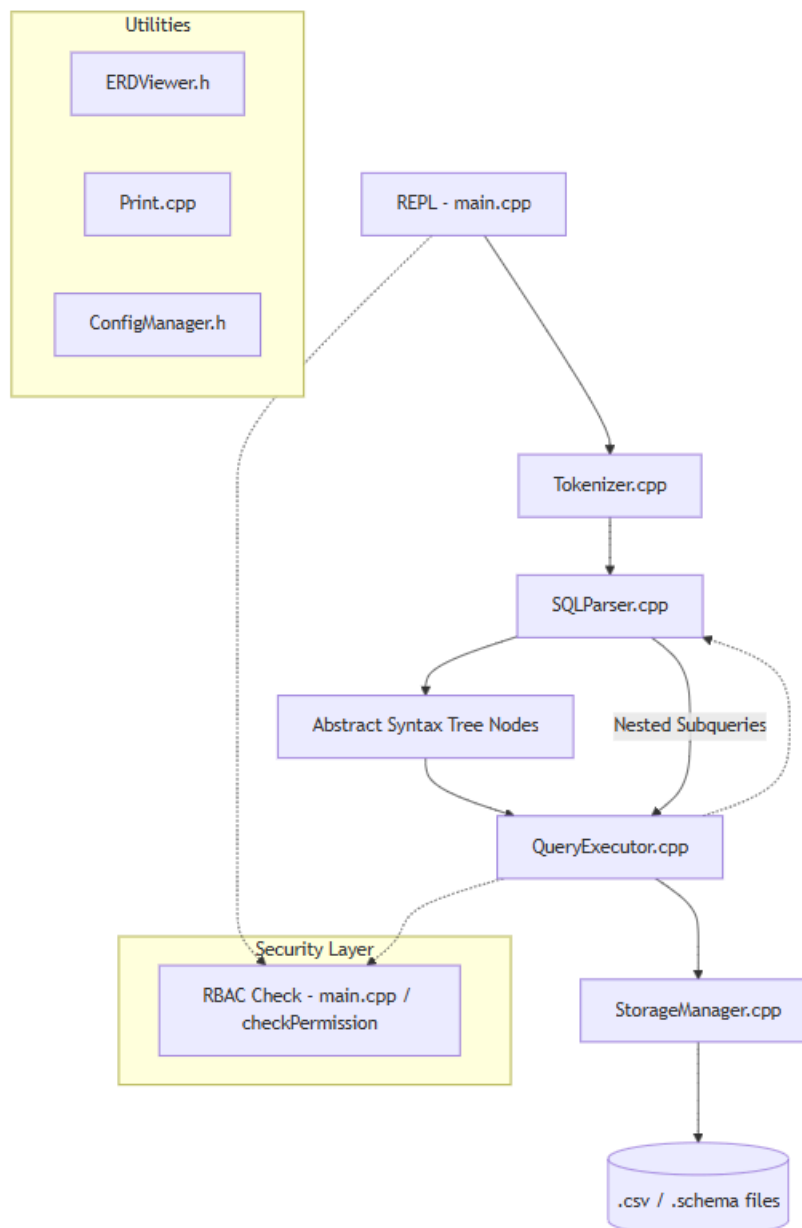
Data persists across sessions through CSV and schema files on disk, which meant we had to think about what "saving state" actually means for a database, not just writing rows, but preserving the schema, the constraints, the roles. None of this is how a real database works at scale. But all of it is how a real database works in principle, and that was the point.

Project Architecture and Flow

FeatherDB utilizes a decoupled, modular pipeline architecture. Every command flows through a strict verification and execution sequence:

REPL → Tokenizer → SQLParser → AST → QueryExecutor → StorageManager → CSV Files

- **REPL:** Captures user input and handles meta-commands (e.g., `.help`, `.tables`, `.erd`).
- **Parser & AST:** A recursive descent parser that validates syntax and builds a hierarchical tree representing the query.
- **Storage Manager:** Acts as the persistence layer, performing CRUD operations on physical `.csv` and `.schema` files in the `/db` directory.



Implemented Features

Core SQL Operations

- **CREATE TABLE** – Defines a new table schema with column types and constraint declarations. The schema is written to disk immediately and used to validate all subsequent operations on that table.
- **INSERT** – Adds a new row after running it through the full constraint pipeline – primary key uniqueness, unique column checks, and foreign key existence – before any write reaches disk.
- **SELECT** – Retrieves data with support for column projection, **WHERE** filtering, nested subqueries in both the **FROM** and **WHERE** clauses, and **ORDER BY** sorting.
- **UPDATE** – Locates rows matching a **WHERE** condition and modifies the specified columns, re-checking uniqueness and referential constraints on the new values before saving.
- **DELETE** – Removes matching rows, but refuses the operation entirely if any other table holds a foreign key reference to the target row (RESTRICT policy).
- **DROP TABLE** – Permanently removes both the data file and the schema file for a table from the **db/** directory.

Constraint Enforcement

- **PRIMARY KEY** – Enforces uniqueness and non-nullability on the designated column. Only one primary key per table is permitted.
- **UNIQUE** – Extends uniqueness guarantees to any column independently of the primary key. Multiple unique columns per table are supported.
- **FOREIGN KEY (REFERENCES)** – Links a column to the primary key of another table. A RESTRICT deletion policy is enforced – a parent row cannot be deleted while child rows still reference it.
- **Automatic Validation** – All three constraint types are checked on every DML operation before any write is committed to disk.

Advanced Query Features

- **Subqueries in FROM** – A nested `SELECT` in the `FROM` clause is treated as a derived table. The inner query is parsed and executed recursively before the outer query runs against its result.
- **Subqueries in WHERE** – A nested `SELECT` in the `WHERE` clause must resolve to a scalar value (one row, one column), which is then substituted back into the condition for evaluation.
- **Multi-level Nesting** – Both subquery forms support multiple levels of recursion, bounded by available call stack depth.
- **ORDER BY** – Results are sorted in-memory using a quicksort implementation applied to the result set before output. Ascending and descending order are both supported.
- **Comparison Operators** – Full support for `=`, `<`, `>`, `<=`, `>=`, and `!=` in `WHERE` clauses.

Role-Based Access Control (RBAC)

- **Authentication** – Roles are activated by entering a secret key at the REPL prompt. The active role is reflected in the prompt display (e.g., `featherDb> (admin)`).
- **Role Creation** – Admins can define new roles with custom secret keys using `CREATE ROLE ... WITH SECRET`.
- **Permission Granting** – Table-level privileges (`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CREATE`, `DROP`) can be assigned independently per role per table using `GRANT`.
- **Session Management** – The active role is tracked throughout the session and checked before every query reaches the execution stage.
- **Persistent Storage** – Role definitions and permission mappings are persisted to `roles.csv` and `permissions.csv` in the `db/` directory.

Relationship Visualization

- **ERD Generation** – The `.erd` meta-command analyzes all schema files and renders a text-based entity-relationship diagram in the terminal. Relationship types are inferred automatically from constraint metadata: a foreign key that is also `UNIQUE` or `PRIMARY KEY` is classified as 1:1; a plain foreign key as 1:N; a table where every column is a foreign key is treated as a junction table representing an N:N relationship.

Data Persistence

- **Two-File Storage Format** – Each table is stored as a `.csv` file for row data and a `.schema` file for column definitions and constraint metadata.
- **Immediate Persistence** – Every write operation is committed to disk without buffering. There is no staging layer or rollback mechanism.
- **Meta-Commands** – `.tables` lists all tables currently stored in the `db/` directory; `.schema <name>` reconstructs the `CREATE TABLE` statement from the schema file.

Tools & Technology

Programming Language

- **C++ (C++17)** – The engine is implemented entirely in C++. The choice was partly pragmatic; the team came from a C background and was already comfortable with explicit memory management and low-level data structure work, and partly intentional. Upon advice of the supervisor, we fixated on building a database in a language without a garbage collector, which meant that decisions around memory ownership and data layout had to be made consciously rather than delegated to a runtime.

Development Environment

- **Visual Studio Code** – Used as the primary editor throughout development. The C/C++ extension handled IntelliSense and debugging; the integrated terminal was used to run the compile script and test queries directly.

Build System

- **Custom Batch Scripts** – The project is compiled via `compile.bat`, which invokes G++ with the appropriate source files and flags. A separate `run_tests.bat` script runs the test case suite defined in `commands.txt`. No external build system (CMake, Make) was used. The scripts are short enough to read and understand in under a minute.

Version Control

- **Git / GitHub** – Git was used for version control across the three-member team. The repository is hosted on GitHub and still serves as the central point for coordinating contributions, tracking changes, and maintaining a recoverable project history.

Standard Library

- **C++ STL** – The implementation relies almost entirely on the standard library. `std::vector`, `std::map`, and `std::set` handle row collections, column metadata, and permission mappings. `std::string` is used throughout for token manipulation and value representation. File I/O uses standard `std::ifstream` and `std::ofstream` streams. No third-party libraries were introduced. An external dependency was only worth considering if the functionality genuinely couldn't be built from scratch, which in practice barely came up.

Diagrams & Presentation

- **Mermaid** – Used to produce architecture and data-flow diagrams. Diagrams are written as text and rendered from within the documentation, keeping them version-controllable alongside the code.
- **Canva** – Used for preparing project presentations.

Application Domain

Potential Use Cases

- **Educational Tool for Database Internals** – The primary use case FeatherDB was built for. Rather than treating SQL as a black box, the codebase exposes every stage of query processing, from tokenization through to disk writes (or files in our case), in a form that can actually be read and understood. It is particularly suited to students who have some programming background but have never looked inside a database engine before.
- **Teaching Relational Concepts** – FeatherDB can be used in classroom settings to demonstrate constraint enforcement, referential integrity, and relational algebra in a working system rather than through abstract examples. The ERD viewer makes schema relationships visible without requiring any external tooling.
- **Lightweight Personal Projects** – For small-scale applications that need basic SQL functionality without the overhead of setting up a full database server, FeatherDB's file-based storage and simple REPL interface are practical enough. It is not suitable for anything beyond low-volume, single-user use, but for prototyping or local tooling, it works fine.
- **Demonstrating RBAC Patterns** – The multi-role authentication and permission system can serve as a reference implementation for understanding how access control is layered into a database engine, which is a concept that often gets glossed over in introductory courses.

Target Audience

- CS students taking database or systems courses who want to see how SQL works under the hood
- Educators looking for a readable, self-contained codebase to walk through in lectures
- Developers who are curious about DBMS internals and want something small enough to explore in a weekend

Non-Target Applications

- Production systems of any kind, FeatherDB has no concurrency control, no transaction support, and no crash recovery
- Any dataset that does not fit comfortably in RAM
- Applications requiring ACID compliance, distributed operation, or high-throughput query processing

Project Timeline

The project spans fourteen weeks in total. The work plan is structured to ensure steady progress from foundational design to full implementation, testing, and presentation. Each phase contributes directly to the core objective of constructing a minimalist, C++-based SQL-like database engine with custom parsing, storage, and execution components.



In the final stretch, we enhanced system security by implementing a passkey-protected Role-Based Access Control (RBAC) system for managing administrative and user permissions. This was followed by a rigorous validation phase where we refined the parser to accurately handle multi-word arguments and enforce semicolon termination. Additionally, we expanded the engine's logical depth by enabling the **WHERE** clause to process essential mathematical comparisons such as `=`, `<`, `>`, `>=`, and `<=`.

Weeks 1-2: Planning and Team Formation

The first two weeks were dedicated to establishing the project's direction through brainstorming sessions, reviewing expectations, discussing feasibility, and forming the project team. During this phase, the team finalized the goal of building a lightweight database engine with minimal external library usage and clarified individual responsibilities.

Weeks 3–4: Proposal Development and Initial Setup

These weeks focused on preparing the project proposal and presenting it to the instructors. The feedback received during this stage guided refinements to the project scope, vocabulary used in parser design, and the plan to transition from Python to C++.

In parallel, the team began setting up the initial project structure in C++, organizing modules, selecting coding conventions, and preparing the environment for test-driven development.

Weeks 5–6: Core Fundamentals and Foundational Components

This stage is dedicated to laying the technical foundation of the system. The team will implement the core utilities required for a minimal database engine, including:

1. Custom string handling utilities (if required by the final design),
2. Lexing/tokenization logic,
3. Preliminary interfaces for parser modules,
4. Initial abstractions following object-oriented principles (e.g., Token, Node, StorageManager interfaces).

The goal of this phase is to ensure that subsequent layers (storage and parsing) have a stable, well-defined base.

Weeks 7–8: Storage Engine and Parser Construction

The storage subsystem will be developed during this period, focusing on file-based persistence, record formatting, and basic table operations.

Alongside the storage layer, the team will work on the parser—constructing the Abstract Syntax Tree (AST) with at least three levels of nesting, refining syntactic rules, and establishing the link between parsed queries and upcoming execution logic.

A ***mid-project progress presentation*** is planned between Weeks 8–9 to evaluate alignment with expectations and receive additional feedback.

Weeks 9–10: Command-Line Interface (CLI) Development

These weeks are devoted to building the interactive command-line interface that users will interact with.

Tasks include:

- Designing input handling and communication with the parser,
- Providing meaningful error messages,
- Integrating the parser with the storage and execution pipeline,
- Ensuring smooth round-trip execution of simple queries.

This phase turns the internal modules into a usable tool.

Weeks 11–12: Extended Features and Role-Based Access Control

After the MVP becomes functional, focus shifts toward the additional features recommended during the proposal feedback. These include:

- designing a role-based access control (RBAC) system, such as admin/user roles with permission boundaries;
- supporting more complex SQL-like operations (e.g., nested queries aligned with the three-level AST requirement);
- strengthening internal modularity and improving parser vocabulary and error handling.

These enhancements expand both the realism and academic value of the project.

Weeks 13–14: Testing, Documentation, and Final Presentation

The final two weeks concentrate on rigorous testing following the test-driven development approach. Unit tests, integration tests, and behavior tests will verify correctness across all components—lexing, parsing, storage, CLI, and RBAC.

Simultaneously, the team will prepare project documentation, including design explanations, module descriptions, usage guidelines, and analysis of limitations.

The work concludes with the final project presentation, demonstrating the database engine's features, architecture, and evolution from proposal to implementation.

Action Based on Feedback (Progress & Evolution)

During the proposal presentation, feedback from the course supervisors and judges led to several concrete changes to the project scope and implementation:

- **Transition to C++** – The original proposal planned to use Python, primarily because it would have been quicker to get something working. Following feedback from the judges, the implementation was shifted to C++. This turned out to be the right call since working closer to the hardware made the internal mechanics of the database more visible, which aligned better with what the project was trying to demonstrate.
- **RBAC Integration** – One of the judges suggested adding an access control layer to make the project more complete as a database system. This led to the implementation of the secret-key authentication system and the role-permission model, which became one of the more interesting parts of the project to build.
- **ERD Visualization** – A judge recommended including a way to visualize table relationships, which resulted in the `.erd` utility. Implementing the relationship detection logic and distinguishing 1:1, 1:N, and N:N from constraint metadata alone ended up being a worthwhile addition.
- **Mathematical Foundations** – The project supervisor advised incorporating a formal treatment of the underlying concepts. This shaped how the project addresses the theoretical basis of the system, grounding the implementation in relational algebra and constraint theory rather than treating it as purely a coding exercise.

Contribution

Collaborative Achievements

All three members participated in design discussions throughout the project, collectively deciding on the system architecture, constraint enforcement strategy, and RBAC structure. Internal documentation, the project README, and the report were developed collaboratively. The build system, coding conventions, and REPL integration were finalized as a group, as were code reviews conducted in preparation for the presentation. QueryExecution and Debugging were also collaborative efforts.

Individual Roles

Md. Saif-AI Sarker (230042106): Testing & Query Execution

Contributed to the query execution layer and was responsible for the project's testing infrastructure. This included unit tests for the Tokenizer, Parser, and AST validation, integration tests for the QueryExecutor covering constraint scenarios, and storage persistence tests verifying CSV read/write correctness. Test automation was set up via `run_tests.bat`, and reproducible bug cases were documented for debugging purposes. Also contributed to the implementation of core DML operations within the QueryExecutor.

M Safwan Hasan Khan (230042117): Storage Engine & Utilities

Contributed to the persistent storage layer and the supporting utilities. This covers the StorageManager (`StorageManager.cpp/h`), which handles CSV serialization and deserialization, schema file management, and data synchronization. Supporting utilities include `ConfigManager.h` for environment-based configuration, `Print.cpp/h` for formatted query output, and the ERD Viewer (`ERDViewer.h`), which automatically detects and visualizes 1:1, 1:N, and N:N relationships from schema metadata.

Wasi Omar Syed (230042143): SQL Parser & Query Execution

Contributed to the SQL processing pipeline, including the Tokenizer (`Tokenizer.cpp/h`), which performs lexical analysis and classifies SQL input into token streams, and the SQLParser (`SQLParser.cpp/h`), a recursive descent parser that constructs Abstract Syntax Trees from tokenized input and handles nested subqueries in both `FROM` and `WHERE` clauses. Also contributed to the QueryExecutor alongside the rest of the team, particularly around expression evaluation and ORDER BY sorting.

Future Work

The current implementation covers the core of what a relational database needs to do, but there is plenty of room to grow. The most natural next step would be adding **JOIN** support. Right now multi-table queries require subquery workarounds, and a proper **INNER JOIN** syntax would make FeatherDB significantly more expressive. Closely related is aggregate function support (**COUNT**, **SUM**, **AVG**, **MIN**, **MAX** with **GROUP BY**), which is probably the most commonly missed feature when working with the current system.

On the SQL completeness side, transaction support with **ROLLBACK** would be a meaningful addition. As of now, every operation commits immediately, which means there is no way to undo a sequence of changes if something goes wrong midway. Views, triggers, and stored procedures, etc., would push FeatherDB further toward a complete SQL environment, though those are substantially more complex to implement cleanly.

The biggest architectural limitation is performance. Every query currently does a full table scan with no indexing, and every operation reloads the table from disk. Introducing a B-tree index structure and a basic buffer pool would address both of these, and would also make the codebase more interesting from a learning perspective; those are concepts that come up in every database course but rarely get implemented from scratch.

Further out, multi-threading and concurrent session support would require rethinking the storage layer significantly, but would make FeatherDB viable for slightly more realistic use cases beyond single-user exploration.

Conclusion

FeatherDB started as a question we couldn't answer from reading slides: What actually happens when you run a SQL query? By the end of it, we had built something that could answer that question in code. A tokenizer, a parser, an AST, an executor, a constraint engine, a security layer, and a visualization tool, all written from scratch in C++.

This was our first proper software project; third semester, first time building something with moving parts that all had to work together. There were decisions we got wrong and had to redo, features that turned out harder than expected, and a few bugs that took embarrassingly long to track down. But the three of us, along with our supervisor, got through it, and the experience of actually building a system rather than just studying one made the underlying concepts click in a way that coursework alone hadn't. We used inspirations from YouTube, GitHub repos like Build-Your-Own-X, AI tools, etc, during development, not to copy code we didn't understand, but as a resource for working through design decisions, understanding tradeoffs, and debugging logic. The implementation and the understanding behind it are genuinely ours.

FeatherDB is not a production database and was never meant to be. What it is though is a readable, working demonstration of how a relational database engine fits together, and for what we set out to learn, that's exactly what it needed to be.

Source Repository

GitHub Link: <https://github.com/SaifSarker11/FeatherDB>